

1. Descripción de la solución

El ejercicio consiste en desarrollar una aplicación de Windows Forms capaz de generar contraseñas de una longitud indicada por el usuario y posteriormente comprobar si la contraseña generada cumple determinadas condiciones para considerarse fuerte.

Para resolverlo se aplicó programación orientada a objetos, separando la lógica de la contraseña en una clase llamada Password y utilizando el formulario únicamente para recibir datos, ejecutar los métodos de la clase y mostrar los resultados.

2. Clase implementada: Password

La clase Password contiene los datos y comportamientos relacionados con una contraseña. Sus atributos son privados, por lo que se controla el acceso a ellos mediante métodos.

Elemento	Tipo	Función
longitud	int	Guarda la cantidad de caracteres de la contraseña.
contraseña	string	Guarda la contraseña actual.
Password()	constructor	Inicializa longitud en 8 y contraseña en 12345678.
getLongitud()	int	Devuelve la longitud actual.
getContraseña()	string	Devuelve la contraseña actual.
setLongitud()	void	Permite establecer la longitud indicada por el usuario.
generarPassword()	void	Genera una contraseña aleatoria usando letras y números.
esFuerte()	bool	Cuenta mayúsculas, minúsculas y números y determina si es fuerte.

3. Código de la clase Password

```
using System;
using System.Collections.Generic;
using System.Text;

namespace pre
{
    internal class Password
    {
        private int longitud;
        private string contraseña;

        public Password()
        {
            longitud = 8;
            contraseña = "12345678";
        }

        public int getLongitud()
        {
            return longitud;
        }
    }
}
```

```

public string getContraseña()
{
    return contraseña;
}

public void setLongitud(int longitud)
{
    this.longitud = longitud;
}

public void generarPassword()
{
    string caracteres = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789";

    Random random = new Random();

    contraseña = "";

    for (int i = 0; i < longitud; i++)
    {
        int posicion = random.Next(caracteres.Length);

        contraseña += caracteres[posicion];
    }
}

public bool esFuerte()
{
    int mayusculas = 0;
    int minusculas = 0;
    int numeros = 0;

    foreach (char caracter in contraseña)
    {
        if (char.IsUpper(caracter))
        {
            mayusculas++;
        }
        else if (char.IsLower(caracter))
        {
            minusculas++;
        }
        else if (char.IsDigit(caracter))
        {
            numeros++;
        }
    }

    return mayusculas > 2 && minusculas > 1 && numeros > 5;
}
}

```

4. Lógica de generación de la contraseña

Paso	Proceso
1	El usuario selecciona la longitud mediante el control NumericUpDown.
2	El formulario obtiene ese valor y lo envía a setLongitud().
3	generarPassword() crea una cadena de caracteres permitidos: A-Z, a-z y 0-9.
4	Se utiliza Random para seleccionar posiciones aleatorias dentro de esa cadena.
5	El ciclo for se repite tantas veces como indique la longitud.
6	La contraseña generada se muestra en el TextBox txtPassword.

5. Lógica para verificar si la contraseña es fuerte

El método esFuerte() recorre cada carácter de la contraseña utilizando foreach. Para cada carácter se comprueba si es una letra mayúscula, una letra minúscula o un número. Al finalizar se evalúa la condición definida en el ejercicio.

Condición utilizada: más de 2 mayúsculas, más de 1 minúscula y más de 5 números.

Por lo tanto, la contraseña se considera fuerte únicamente cuando cumple las tres condiciones al mismo tiempo. Si alguna no se cumple, el resultado es CONTRASEÑA DÉBIL.

6. Código del formulario Form1.cs

```
namespace pre
{
    public partial class Contraseñas : Form
    {
        Password password = new Password();

        public Contraseñas()
        {
            InitializeComponent();

            nudLongitud.Value = password.getLongitud();
            txtPassword.Text = password.getContraseña();
        }

        private void btnGenerar_Click(object sender, EventArgs e)
        {
            int longitud = (int)nudLongitud.Value;

            password.setLongitud(longitud);

            password.generarPassword();

            txtPassword.Text = password.getContraseña();
        }

        private void btnVerificar_Click(object sender, EventArgs e)
        {
            if (password.esFuerte())
            {
                txtresultado.Text = "CONTRASEÑA FUERTE";
            }
            else
            {
                txtresultado.Text = "CONTRASEÑA DÉBIL";
            }
        }
    }
}
```

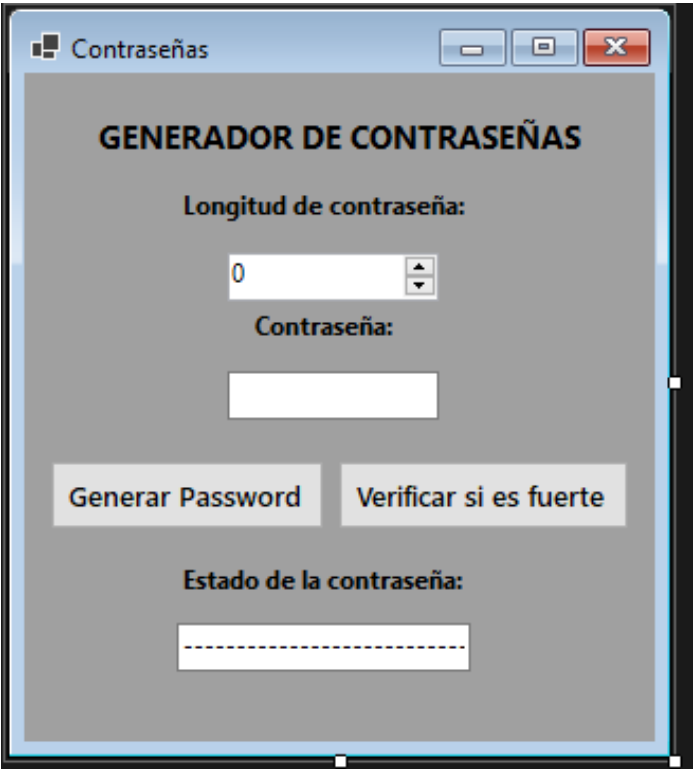
```

    }
}
}

```

7. Interfaz gráfica

La interfaz fue diseñada en Windows Forms. Permite seleccionar la longitud, generar una contraseña, verificar su fortaleza y mostrar el estado obtenido.



8. Elementos utilizados en la interfaz

Nombre	Control	Uso
nudLongitud	NumericUpDown	Seleccionar la longitud de la contraseña.
txtPassword	TextBox	Mostrar la contraseña generada.
btnGenerar	Button	Generar una nueva contraseña.
btnVerificar	Button	Comprobar si la contraseña es fuerte.
txtresultado	TextBox	Mostrar CONTRASEÑA FUERTE o CONTRASEÑA DÉBIL.

9. Corrida de la solución

Para demostrar el funcionamiento se puede seleccionar una longitud, presionar Generar Password y después presionar Verificar si es fuerte.

Dato	Ejemplo de corrida	Resultado
Longitud	11	Se generan 11 caracteres.
Contraseña generada	ABCde123456	Contiene 3 mayúsculas, 2

		minúsculas y 6 números.
Verificación	esFuerte()	true
Estado mostrado	CONTRASEÑA FUERTE	La contraseña cumple las tres condiciones.

Nota: debido a que la generación utiliza valores aleatorios, una contraseña generada automáticamente no necesariamente será fuerte en cada intento. El botón de verificación evalúa la contraseña que se encuentre actualmente en txtPassword.

10. Conclusión

La solución implementa correctamente una clase Password para encapsular la información y las operaciones de una contraseña. El formulario interactúa con dicha clase mediante sus métodos, respetando la separación entre la lógica del programa y la interfaz gráfica.

Con este ejercicio se aplicaron conceptos de programación orientada a objetos como clases, atributos privados, constructor, métodos get y set, generación aleatoria, ciclos, condicionales y valores booleanos.

11. Corrida de la aplicación

La siguiente captura corresponde a una ejecución real del programa. En este caso, se seleccionó una longitud de 8 caracteres y el sistema generó la contraseña "iQiEeEmY". Al presionar el botón "Verificar si es fuerte", el programa muestra "CONTRASEÑA DÉBIL", ya que la contraseña no cumple simultáneamente con los requisitos de más de 2 mayúsculas, más de 1 minúscula y más de 5 números.

Contraseñas

GENERADOR DE CONTRASEÑAS

Longitud de contraseña:

8

Contraseña:

iQiEeEmY

Generar Password Verificar si es fuerte

Estado de la contraseña:

CONTRASEÑA DÉBIL